

APLIKACJE WEBOWE

Branching & Team Workflows

L4



by ***Mateusz Węglewski***

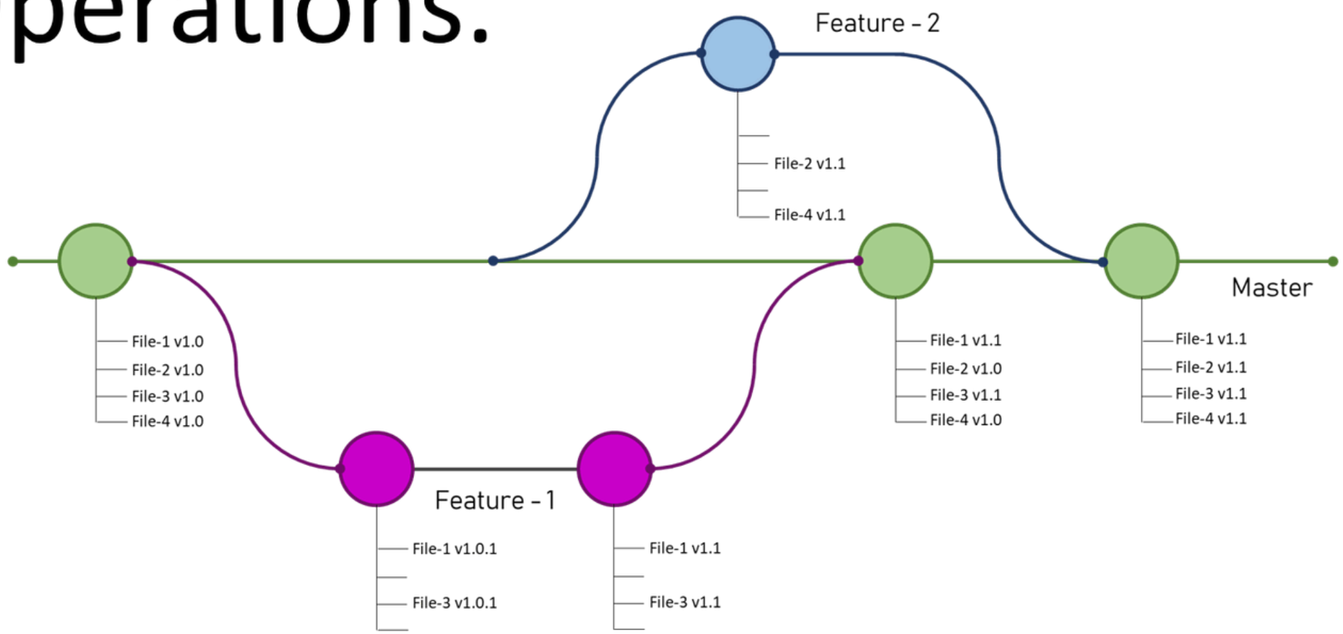


Branch

A **branch** in Git is simply a **lightweight movable pointer to one of the commits.** The default branch name in Git is *master* . As you start making commits, you're given a master branch that points to the last commit you made. Every time you commit, the master branch pointer moves forward automatically.



GIT Branch and its Operations.



by **Mateusz Węglewski**



APLIKACJE WEBOWE

Po co nam branchowanie?



by ***Mateusz Węglewski***



APLIKACJE WEBOWE

branch	wskaźnik na commit; niezależna linia rozwoju kodu, którą można tworzyć i usuwać bez ryzyka
HEAD	wskaźnik na commit, na którym aktualnie 'stoisz' — zwykle czubek bieżącej gałęzi
`git switch <gałąź>`	przełącza katalog roboczy na wskazaną gałąź
working directory	pliki na dysku, które widzisz i edytujesz — zmieniają się przy przełączaniu gałęzi





Merge

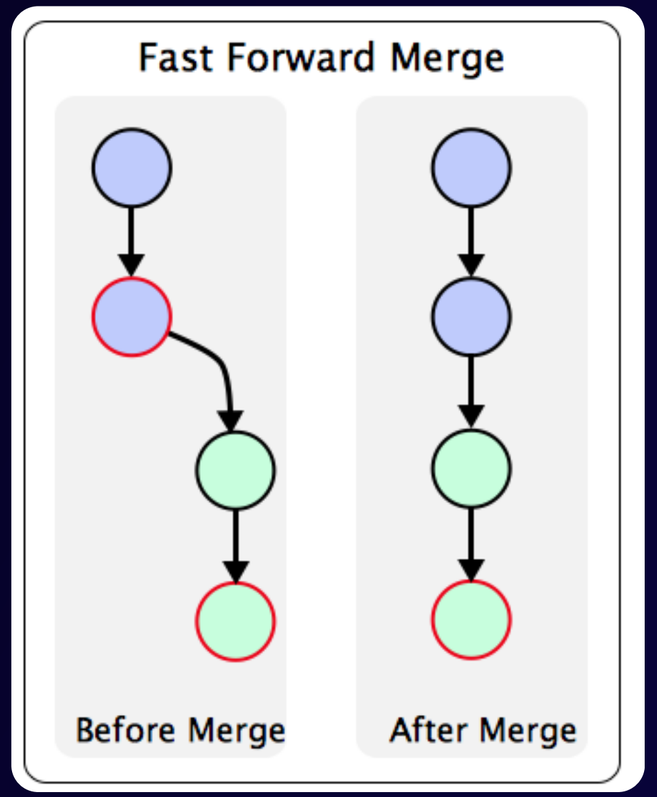
git merge is a Git command used to combine changes from two different development lines (called branches) into a single branch.
Think of it like two people working on different chapters of the same book; merging is the act of putting those chapters back together into the main manuscript.



Fast-forward merge

Fast-forward merge to **najprostszy i najczystszy** sposób łączenia gałęzi w Git. Występuje **automatycznie**, **gdy historia między dwoma branchami nie uległa rozwidleniu.**

Mówiąc najprościej: dzieje się tak wtedy, gdy od momentu utworzenia Twojej gałęzi bocznej (np. feature), na gałęzi głównej (main) nie pojawiły się żadne nowe commity.



by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Fast-forward merge

```
git switch -c feature/uppercase
# ...zmiany + commit...
git switch main
git merge feature/uppercase

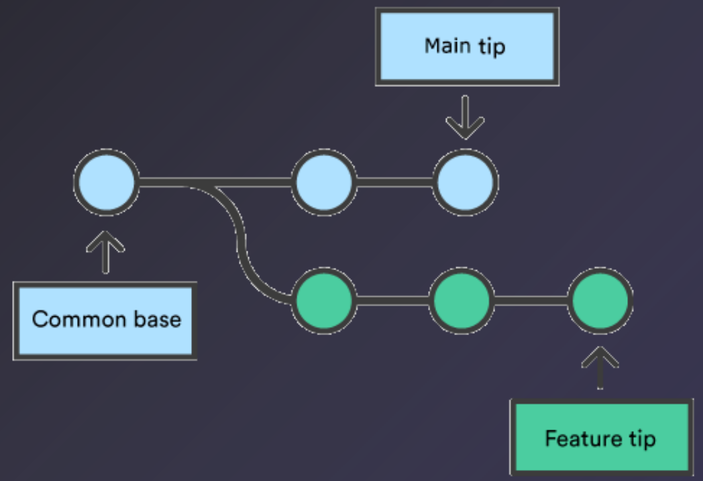
Updating 3a1f9c2..7b8e401
Fast-forward
```

Main nie miał żadnych nowych commitów od momentu utworzenia gałęzi — Git po prostu przesuwa wskaźnik main do przodu. Brak dodatkowego commitu.



3-way merge (merge commit)

Three-way merge to sytuacja, w której Git tworzy tzw. **merge commit (commit scalenia)**. Dzieje się tak **automatycznie**, **gdy historie obu gałęzi się rozeszły** - czyli od momentu utworzenia gałęzi bocznej, na gałęzi głównej również pojawiły się nowe zmiany.



by ***Mateusz Węglewski***



APLIKACJE WEBOWE

3-way merge

```
git switch main
# ...ktoś inny dodał commit na main...
git merge feature/greeting

Merge made by the 'recursive' strategy.
Merge branch 'feature/greeting'
```

Main ma już nowe commity, których nie ma na feature - Git tworzy dodatkowy commit scalający z dwoma rodzicami, łącząc obie historie.





APLIKACJE WEBOWE

✕ Fast-forward

main bez nowych commitów

→ wskaźnik przesuwa się, brak nowego commitu

✓ 3-way merge

main ma nowe commity

→ powstaje dodatkowy merge commit z dwoma rodzicami

by *Mateusz Węglewski*



APLIKACJE WEBOWE

Git flow – ścisłe role for each branch

main / develop	**main = zawsze gotowe do wydania; develop = integracja bieżącej pracy**
*feature/**	**rozwój pojedynczej funkcjonalności, odgałęziona i scalana z powrotem do develop**
*release/**	**przygotowanie konkretnej wersji (numeracja, changelog) przed wydaniem**
*hotfix/**	**pilna poprawka bezpośrednio na produkcji, odgałęziona z main**

Najlepsze dla: większych projektów z zaplanowanym cyklem wydań.



GitHub Flow – jeden main branch, zawsze gotowy do wdrożenia

1. Utwórz gałąź z main o opisowej nazwie
2. Wprowadzaj commity, pushuj regularnie
3. Otwórz Pull Request, gdy gotowe (lub wcześniej, jako draft)
4. Poproś o code review
5. Scal do main po akceptacji
6. Wdróż main natychmiast po scaleniu

Najlepsze dla: zespołów wdrażających ciągle, dowolnej wielkości.

by ***Mateusz Węglewski***

Trunk-based development – codzienna integracja do jednego brancha

trunk / main	**praktycznie jedyna długotrwała gałąź w repozytorium**
short-lived branches	**jeśli w ogóle istnieją, żyją nie dłużej niż dzień**
feature flags	**niedokończona praca trafia do main, ale jest ukryta za przełącznikiem**
integracja ≥ 1×/dzień	**każdy deweloper scala swoją pracę do trunk co najmniej raz dziennie**

Najlepsze dla: zespołów o wysokiej dojrzałości CI/CD i silnym pokryciu testami.

by ***Mateusz Węglewski***



Strategia	Struktura gałęzi	Najlepsze dla
Git Flow	main + develop + feature/release/hotfix	**duże projekty, zaplanowane wydania**
GitHub Flow	main + krótkie gałęzie feature	**ciągłe wdrożenia, dowolna wielkość zespołu**
Trunk-Based Dev.	głównie trunk, branch max 1 dzień	**wysoka dojrzałość CI/CD, silne testy**



Jak nazywać branche?

feature/<opis>	nowa funkcjonalność, np. feature/shopping-cart
fix/<opis>	poprawka błędu niebędąca pilną sytuacją produkcyjną, np. fix/date-validation
hotfix/<opis>	pilna poprawka błędu na produkcji, np. hotfix/payment-crash
release/<wersja>	przygotowanie konkretnego wydania, np. release/2.3.0

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

✕ Bad

```
"mybranch123"

"fix"

"test"
```

✓ Good

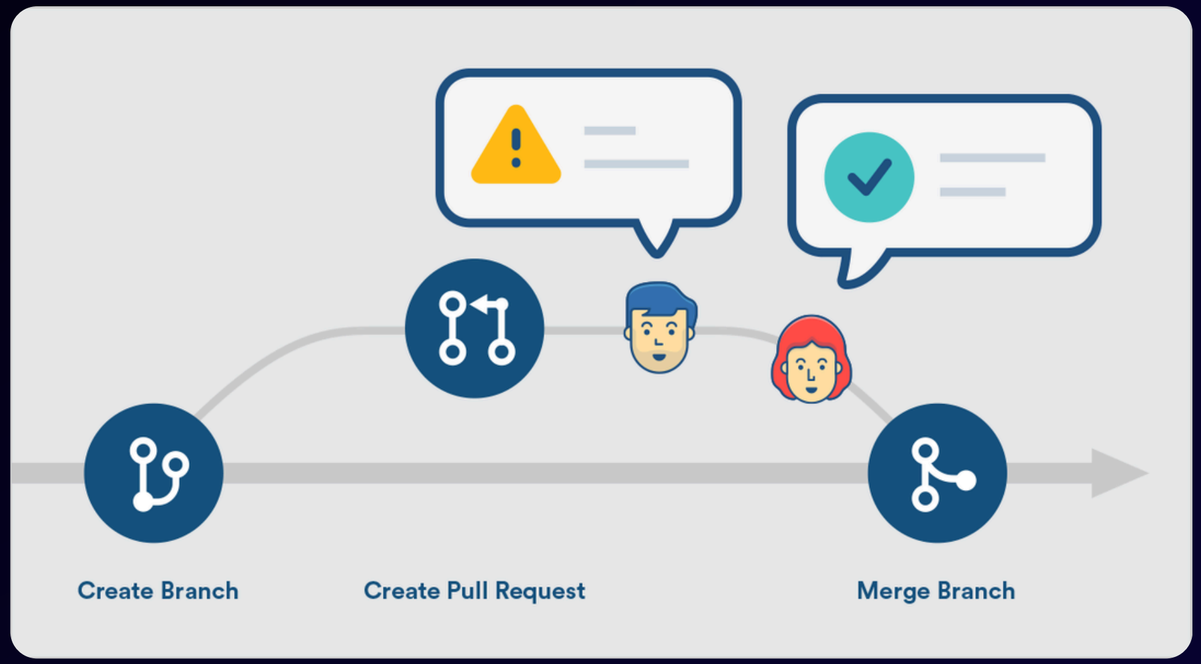
```
"feature/
shopping-cart"

"hotfix/
payment-crash"
```




pull request (PR)

prośba o przejrzenie i scalenie zmian z jednej gałęzi do drugiej — centralne miejsce dyskusji o kodzie



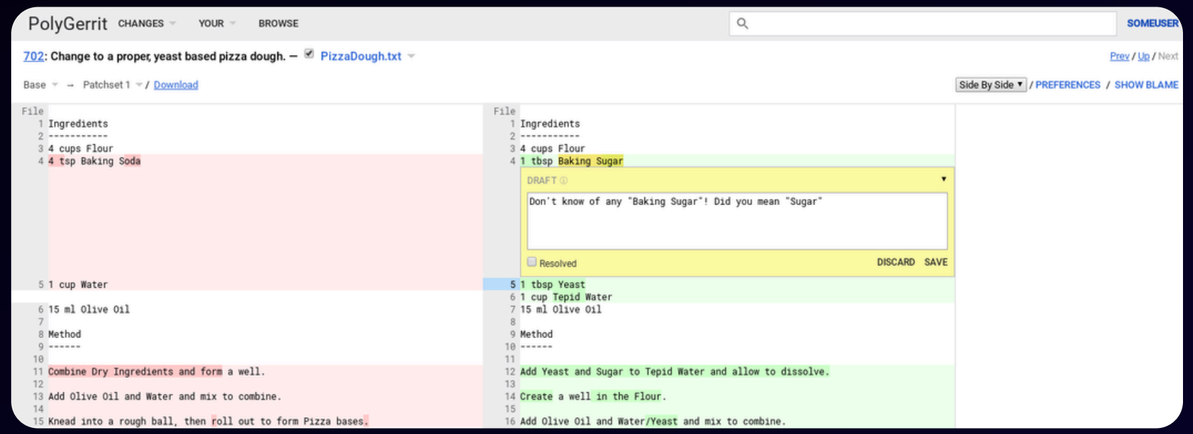
by **Mateusz Węglewski**

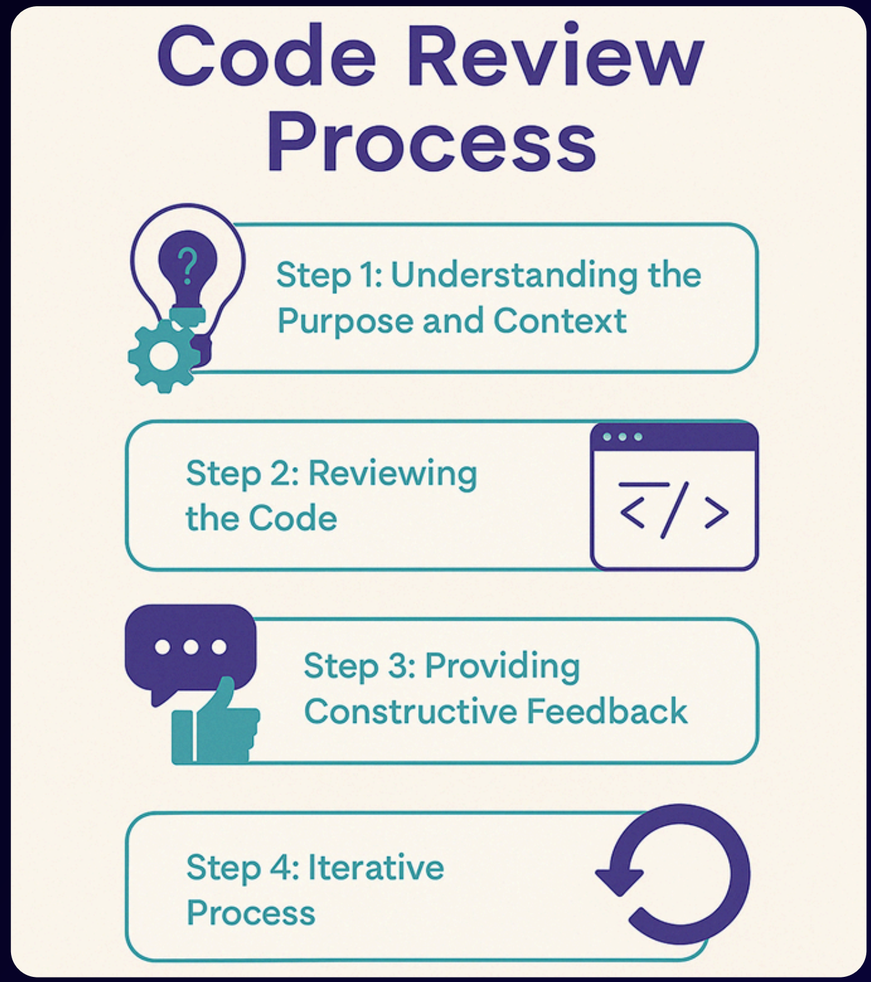


APLIKACJE WEBOWE

code review

proces sprawdzania pull requestu przez innego członka zespołu przed scaleniem





by ***Mateusz Węglewski***

Od gałęzi do wdrożenia (GitHub Flow)



main jest zawsze gotowy do wdrożenia - dlatego wdrożenie następuje natychmiast po scaleniu.

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Polecenie	Co robi
`git clone <url>`	tworzy lokalną kopię repozytorium
`git branch`	wyświetla listę gałęzi
`git branch <nazwa>`	tworzy nową gałąź (bez przełączania)
`git switch <nazwa>`	przełącza na istniejącą gałąź
`git switch -c <nazwa>`	tworzy nową gałąź i od razu na nią przełącza
`git status`	pokazuje stan repozytorium i zmian
`git add . / git add <plik>`	dodaje zmiany do staging area
`git commit -m "..."`	zapisuje zmiany jako commit
`git diff <gałąź-A> <gałąź-B>`	pokazuje różnicę między dwiema gałęziami
`git merge <nazwa>`	scala wskazaną gałąź do bieżącej
`git branch -d <nazwa>`	usuwa scaloną gałąź
`git log --oneline --graph --all`	wizualizuje historię wszystkich gałęzi
`git init --bare`	tworzy "gołe" repozytorium pełniące rolę zdalnego serwera
`git push origin <gałąź>`	wysyła commity danej gałęzi do zdalnego repo
`git fetch origin`	pobiera informacje o zmianach na zdalnym repo (bez scalania)





APLIKACJE WEBOWE

:wq

L4



by ***Mateusz Węglewski***